generate-tasks

Rule: Generating a Task List from User Requirements

Goal

To guide an AI assistant in creating a detailed, step-by-step task list in Markdown format based on user requirements, feature requests, or existing documentation. The task list should guide a developer through implementation.

Output

- **Format:** Markdown (`.md`)
- **Location:** `/tasks/`
- **Filename:** `tasks-[feature-name].md` (e.g., `tasks-user-profile-editing.md`)

Process

1. **Receive Requirements:** The user provides a feature request, task description, or points to existing documentation
2. **Analyze Requirements:** The AI analyzes the functional requirements, user needs, and implementation scope from the provided information
3. **Phase 1: Generate Parent Tasks:** Based on the requirements analysis, create the file and generate the main, high-level tasks required to implement the feature. **IMPORTANT: Always include task 0.0 "Create feature branch" as the first task, unless the user specifically requests not to create a branch.** Use your judgement on how many additional high-level tasks to use. It's likely to be about 5. Present these tasks to the user in the specified format (without sub-tasks yet). Inform the user: "I have generated the high-level tasks based on your requirements. Ready to generate the sub-tasks? Respond with 'Go' to proceed."
4. **Wait for Confirmation:** Pause and wait for the user to respond with "Go".
5. **Phase 2: Generate Sub-Tasks:** Once the user confirms, break down each parent task into smaller, actionable sub-tasks necessary to complete the parent task. Ensure sub-tasks logically follow from the parent task and cover the implementation details implied by the requirements.
6. **Identify Relevant Files:** Based on the tasks and requirements, identify potential files that will need to be created or modified. List these under the `Relevant Files` section, including corresponding test files if applicable.
7. **Generate Final Output:** Combine the parent tasks, sub-tasks, relevant files, and notes into the final Markdown structure.
8. **Save Task List:** Save the generated document in the `/tasks/` directory with the filename `tasks-[feature-name].md` , where `[feature-name]` describes the main feature or task being implemented (e.g., if the request was about user profile editing, the output is `tasks-user-profile-editing.md`).

Output Format

The generated task list *must* follow this structure:

```
## Relevant Files

- `path/to/potential/file1.ts` - Brief description of why this file is relevant (e.g., Contains the main component for this feature).
- `path/to/file1.test.ts` - Unit tests for `file1.ts`.
- `path/to/another/file.tsx` - Brief description (e.g., API route handler for data submission).
- `path/to/another/file.test.tsx` - Unit tests for `another/file.tsx`.
- `lib/utils/helpers.ts` - Brief description (e.g., Utility functions needed for calculations).
- `lib/utils/helpers.test.ts` - Unit tests for `helpers.ts`.

### Notes
```

```
- Unit tests should typically be placed alongside the code files they are testing (e.g., `MyComponent.tsx` and `MyComponent.test.tsx` in the same directory).
- Use `npx jest [optional/path/to/test/file]` to run tests. Running without a path executes all tests found by the Jest configuration.

## Instructions for Completing Tasks

**IMPORTANT:** As you complete each task, you must check it off in this markdown file by changing `- [ ]` to `- [x]`. This helps track progress and ensures you don't skip any steps.

Example:
- `- [ ] 1.1 Read file` → `- [x] 1.1 Read file` (after completing)

Update the file after completing each sub-task, not just after completing an entire parent task.

## Tasks

- [ ] 0.0 Create feature branch
  - [ ] 0.1 Create and checkout a new branch for this feature (e.g., `git checkout -b feature/[feature-name]`)
- [ ] 1.0 Parent Task Title
  - [ ] 1.1 [Sub-task description 1.1]
  - [ ] 1.2 [Sub-task description 1.2]
- [ ] 2.0 Parent Task Title
  - [ ] 2.1 [Sub-task description 2.1]
- [ ] 3.0 Parent Task Title (may not require sub-tasks if purely structural or configuration)
```

Interaction Model

The process explicitly requires a pause after generating parent tasks to get user confirmation ("Go") before proceeding to generate the detailed sub-tasks. This ensures the high-level plan aligns with user expectations before diving into details.

Target Audience

Assume the primary reader of the task list is a **junior developer** who will implement the feature.